

Database Management Systems: Functional Dependencies & Normalization

Canonical Cover (Minimal Cover)

Definition

A Canonical Cover (also called Minimal Cover) is the smallest equivalent set of Functional Dependencies that represents the same constraints as the original FD set.

It has:

- No redundant FDs.
- No redundant attributes.
- Exactly one attribute on the right-hand side of each FD.

Properties of a Minimal Cover

For an FD set F , its Minimal Cover F_c :

- Every FD has one attribute on the RHS.
- No attribute on the LHS is irrelevant
- No FD is redundant.

F_c is equivalent to F .

Algorithm

Given FD set F :

Step 1

Split RHS attributes.

Step 2

Remove extraneous attributes from the LHS.

Step 3

Remove redundant FDs.

Example

Given

$A \rightarrow BC$
 $B \rightarrow C$
 $A \rightarrow B$
 $AB \rightarrow C$

Step 1

Split RHS

$A \rightarrow B$
 $A \rightarrow C$
 $B \rightarrow C$
 $A \rightarrow B$
 $AB \rightarrow C$

Step 2

Remove duplicates

$A \rightarrow B$
 $A \rightarrow C$
 $B \rightarrow C$
 $AB \rightarrow C$

Step 3

Check redundancy

Since

$A \rightarrow B$
 $B \rightarrow C$

implies

$A \rightarrow C$

Remove $A \rightarrow C$,Now check

$AB \rightarrow C$

Already implied by

$A \rightarrow B$

$B \rightarrow C$

Remove it.

Final Minimal Cover

$A \rightarrow B$

$B \rightarrow C$.

Extraneous Attributes

Definition

An attribute is extraneous if removing it does not change the closure of the FD set.

Example

$AB \rightarrow C$

Suppose

$A \rightarrow C$

already exists.

Then

B

is unnecessary.

So

$AB \rightarrow C$

becomes

$A \rightarrow C.$

Left Extraneous Attribute

Example

$AB \rightarrow D$

If

$A^+ = ABD$

then B is unnecessary.

Right Extraneous Attribute

Example

$$A \rightarrow BC$$

If C is already implied by other FDs,
remove it.

Attribute Closure Algorithm

One of the most important GATE topics.

Definition

The closure of an attribute set X, denoted by X^+ , is the set of all attributes that can be functionally determined from X.

Used for:

- Finding Candidate Keys.
- Testing Functional Dependencies.

Step 1

Start with

$$X^+ = X$$

Step 2

Repeatedly apply FDs.

Whenever

$$\text{LHS} \subseteq X^+$$

add RHS.

Step 3

Repeat until no more attributes can be added.

Candidate Key Algorithm

Definition

A Candidate Key is the smallest attribute set whose closure equals all attributes of the relation.

Algorithm

Step 1

Find attributes never appearing on RHS.

They must be in every Candidate Key.

Step 2

Compute closures.

Step 3

Add attributes until

$X^+ = \text{All attributes}$

Step 4

Remove unnecessary attributes.

Example

Relation

$R(A, B, C, D)$

FDs

$A \rightarrow B$
 $B \rightarrow C$
 $AC \rightarrow D$

Compute

A^+

A

↓

AB

↓

ABC

↓

ABCD

Therefore

Candidate Key

A.

BCNF Decomposition Algorithm

BCNF removes every FD where:

Determinant $\neq$ Candidate Key.

Algorithm

Step 1

Find BCNF violation.

Step 2

Choose violating FD

$X \rightarrow Y$

Step 3

Split into

$R_1(XY)$
 $R_2(R - Y) \cup X$

Repeat until every relation satisfies BCNF.

Example

$\text{StudentID} \rightarrow \text{Department}$
 $\text{Department} \rightarrow \text{HOD}$

Violation

$\text{Department} \rightarrow \text{HOD}$

Department isn't Candidate Key.

Split

Department

Student

Done.

3NF Synthesis Algorithm

Unlike BCNF,

3NF guarantees

Dependency Preservation.

Algorithm

Step 1

Compute Minimal Cover.

Step 2

For every FD

```
X → Y
```

Create one relation

```
XY
```

Step 3

Remove duplicate relations.

Step 4

If no relation contains a Candidate Key,
add one.

Example

FDs

```
StudentID → Name  
StudentID → Department  
Department → HOD
```

Relations

```
Student(StudentID, Name, Department)  
Department(Department, HOD) .
```

Chase Algorithm (Lossless Join Test)

The Chase Algorithm is a formal procedure used to verify whether a decomposition is lossless. It is mainly used in theory,

For a decomposition of R into R_1 and R_2 :

It is lossless if:

$$(R_1 \cap R_2) \rightarrow R_1$$

or

$$(R_1 \cap R_2) \rightarrow R_2$$

using the given FD set.

Example

Relation

$R(A, B, C)$

FDs

$A \rightarrow B$

Decompose into

$R_1(A, B)$

$R_2(A, C)$

Common attribute

A

Since

$A \rightarrow B$

the common attribute determines all attributes of R_1 .

Therefore, the decomposition is lossless.

Dependency Preservation Test

Definition

A decomposition preserves dependencies if every original FD can still be enforced without joining the decomposed relations.

Procedure

For each original FD:

- Check whether its attributes exist within a single decomposed relation.
- If not, compute closures using the projected FD sets.
- If all original FDs can still be derived, the decomposition is dependency preserving.

Original Relation

```
Student(StudentID, Name, Department, HOD)
```

FDs

```
StudentID → Name  
StudentID → Department  
Department → HOD
```

Decompose into

```
Student(StudentID, Name, Department)  
Department(Department, HOD)
```

Every FD is preserved.

Closure of Functional Dependencies (F^+)

Definition

The closure of a Functional Dependency set, denoted F^+ , is the set of all Functional Dependencies that can be derived from the original FD set using Armstrong's Axioms.

Difference Between X^+ and F^+

X^+	F^+
Closure of an attribute set	Closure of an FD set
Produces attributes	Produces Functional Dependencies.

Example

Given

```
A → B
B → C
```

Then

F^+ includes:

```
A → B
B → C
A → C
AB → C
A → BC ,
```

Equivalence of FD Sets

Definition

Two FD sets F_1 and F_2 are equivalent if they imply exactly the same Functional Dependencies. Mathematically:

$$F_1^+ = F_2^+ ,$$

How to Test Equivalence

Step 1

Check whether every FD in F_1 can be derived from F_2 .

Step 2

Check whether every FD in F_2 can be derived from F_1 .